SpiralTrain
developer courses

Appendix

• Generators

- Iteration
- Consuming Iterables
- Iteration Protocol
- Supporting Iteration
- Generators
- Generator Functions
- Conveniant Iterator
- Generator Expression
- Building Blocks
- Programming Problem
- Problem Solution

© copyright : spiraltrain@gmail.com

SpiralTrain developer courses

Iteration

- **Python `for` statement can iterate over many kinds of objects**
- **Iterate over a collection of items :**

```
for x in [1,4,5,10]:
      print (x)   # 1 4 5 10
```

- **Iteration over dictionary given the keys :**

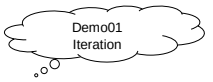
```
prices = { 'GOOGLE' : 490.10, ' IBM' : 145.23, 'YAHOO' : 21.71 }
for key in prices:
     print (key)  # GOOGLE IBM YAHOO
```

- **Iteration over a string gives characters in string :**

```
s = "Mars!"
for c in s:
    print (c)  # M a r s !
```

- **Iteration over a file gives the lines :**

```
for line in open("realprogrammers.txt"):
     print (line)
```



In computer science, an iterator is an object that allows a programmer to traverse through all the elements of a collection regardless of its specific implementation.

Iterators in Python are a fundamental part of the language and in many cases go unseen as they are implicitly used in the for (foreach) statement, in list comprehensions, and in generator expressions.

"Iterators are the secret sauce of Python 3. They're everywhere, underlying everything, always just out of sight. Comprehensions are just a simple form of iterators. Generators are just a simple form of iterators. A function that yields values is a nice, compact way of building an iterator without building an iterator." - Mark Pilgrim

All of Python's standard built-in sequence types support iteration, as well as many classes which are part of the standard library.

The for loop works on any iterable object. Actually, this is true of all iteration tools that scan objects from left to right in Python including for loops, the list comprehensions, and the map built-in function, etc.

Though the concept of iterable objects is relatively recent in Python, it has come to permeate the language's design. Actually, it is a generalization of the sequences. An object is iterable if it is either a physically stored sequence or an object that produces one result at a time in the context of an iteration tool like a for loop.

Consuming Iterables

- **Many functions can consume an iterable object :**
- **Reductions :**

```
s = [1,4,5,10]
print (sum(s))
print (min(s))
print (max(s))
```

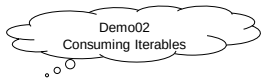
- **Constructors :**

```
l = list(s)
print (l)
t = tuple(s)
print (t)
st = set(s)
print (st)
```

- `in` **operator :**

```
item in s
```

- **And many others in the library**



As a way of understanding the file iterator, we'll look at how it works with a file. Open file objects have readline() method. This reads one line each time we call readline(), we advance to the next line. At the end of the file, an empty string is returned. We detect it to get out of the loop.

```
>>> f = open('C:\\workspace\\Masters.txt')
>>> f.readline()
'Michelangelo Buonarroti \n'
>>> f.readline()
'Pablo Picasso\n'
>>> f.readline()
'Rembrandt van Rijn \n'
>>> f.readline()
'Leonardo Da Vinci \n'
>>> f.readline()
'Claude Monet \n'
>>> f.readline()
'\n'
# Returns an empty string at end-of-file
>>> f.readline()
''
>>>
```

But files also have a __next__() method that has an identical effect. It returns the next line from a file each time it is called. The only difference is that __next__() method raises a built-in StopIteration exception at end-of-file instead of returning an empty string

SpiralTrain developer courses

Iteration Protocol

- **Protocol implemented by objects that you can iterate over :**

```
items = [1, 4, 5]
it = items.__iter__()
print (next(it))
print (next(it))
print (next(it))
print (next(it))
Traceback (most recent call last):
File "D:\..\src\demo03_iteration_protocol.py", line 7, in <module>
   print (next(it))
StopIteration
```

- **Underneath the covers of `for` statement :** `for x in obj:`

```
_iter = ojb.__iter__() # Get iterator object
while 1:
try:
x = next(_iter) # Get next item
except StopIteration: # No more items
break
# statements
```



- **Object supporting `__iter__()` and `__next__()` is "iterable"**

We have a built-in next() function for manual iteration. The next() function automatically calls an object's __next__() method. For an object X, the call next(X) is the same as X.__next__() but simpler.

```
>>> f = open('C:\\workspace\\Masters.txt')
>>> f.__next__()
'Michelangelo Buonarroti \n'
>>> f.__next__()
'Pablo Picasso\n'
>>>
>>> f = open('C:\\workspace\\Masters.txt')
>>> next(f)
'Michelangelo Buonarroti \n'
>>> next(f)
'Pablo Picasso\n'
>>>
```

When the for loop begins, it obtains an iterator from the iterable object by passing it to the iter built-in function. This object returned by iter has the required the __next__() method. Let's look at the internals of this through for loop with lists. For Python versions < 3, we may want to use next(iterObj) instead of iterobj.__next__().

Supporting Iteration

- **User-defined objects can support iteration :**
  - Object implement should implement `__iter__()` and `__next()__`

```
class CountDown(object):
    def __init__(self,start):
        self.count = start
    def __iter__(self):
        return self
    def __next__(self):
        if self.count <= 0:
            raise StopIteration
        r = self.count
        self.count -= 1
        return r
```

- **Now `CountDown` can be iterated as follows :**

```
c = CountDown(5)
for i in c:
    print(i)
```



We've looked at the iterators for files and lists. How about the others such as dictionaries? To step through the keys of a dictionary is to request its keys list explicitly:

```
>>> D = {'a':97, 'b':98, 'c':99}
>>> for k in D.keys():
            print(k, D[k])
a 97
c 99
b 98
```

Dictionaries have an iterator that automatically returns one key at a time in an iteration context:

```
>>> iterObj = iter(D)
>>> next(iterObj)
'a'
>>> next(iterObj)
'c'
>>> next(iterObj)
'b'
>>> next(iterObj)
Traceback (most recent call last):
  File ...
    next(iterObj)
StopIteration
>>>
```

Generators

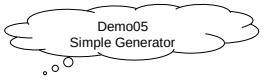
- **Functions producing sequence of results instead of single value :**

```
def countdown(n):
    while n > 0:
        yield n
        n -= 1
for i in countdown(5):
    print (i)
```

- **Instead of returning a value :**
  - Generates series of values using `yield` statement
- **Behavior is quite different from normal function :**
  - Calling a generator function creates an generator object
  - However, it does not start running the function

```
x = countdown(10)
print(x)   # <generator object at 0x58490>
```

- **Notice that no output was produced**



In computer science, a generator is a special routine that can be used to control the iteration behavior of a loop.

A generator is very similar to a function that returns an array, in that a generator has parameters, can be called, and generates a sequence of values. However, instead of building an array containing all the values and returning them all at once, a generator yields the values one at a time, which requires less memory and allows the caller to get started processing the first few values immediately. In short, a generator looks like a function but behaves like an iterator.
Python provides tools that produce results only when needed:

Generator functions

They are coded as normal def but use yield to return results one at a time, suspending and resuming.

Generator expressions

These are similar to the list comprehensions. But they return an object that produces results on demand instead of building a result list.

Because neither of them constructs a result list all at once, they save memory space and allow computation time to be split by implementing the iteration protocol.

Generator Functions: yield vs. return

We can write functions that send back a value and later be resumed by picking up where they left off. Such functions are called generator functions because they generate a sequence of values over time.

Generator functions are not much different from normal functions and they use defs. When created, however, they are automatically made to implement the iteration protocol so that they can appear in iteration contexts.

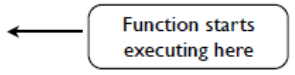
Generator Functions

- **Function only executes on `next()` :**

```
x = countdown(10)
print (x)  # <generator object countdown at 0x00D5B4E0>
```

- `yield` **produces a value, but suspends the function :**
  - Function resumes on next call to `next()`
- **Counting down from 10 :**

```
print (next(x))    #10
print (next(x))    #9
print (next(x))    #8
print (next(x))    #7
print (next(x))    #6
print (next(x))    #5
print (next(x))    #4
print (next(x))    #3
print (next(x))    #2
print (next(x))    #1
print (next(x))    # Exception => Stopiteration
```





Normal functions return a value and then exit. But generator functions automatically suspend and resume their execution. Because of that, they are often a useful alternative to both computing an entire series of values up front and manually saving and restoring state in classes. Because the state that generator functions retain when they are suspended includes their local scope, their local variables retain information and make it available when the functions are resumed.

The primary difference between generator and normal functions is that a generator yields a value, rather than returns a value. The yield suspends the function and sends a value back to the caller while retains enough state to enable the function immediately after the last yield run. This allows the generator function to produce a series of values over time rather than computing them all at once and sending them back in a list.

Generators are closely bound up with the iteration protocol. Iterable objects define a __next__() method which either returns the next item in the iterator or raises the special StopIteration exception to end the iteration. An object's iterator is fetched with the iter built-in function.

The for loops use this iteration protocol to step through a sequence or value generator if the protocol is suspended. Otherwise, iteration falls back on repeatedly indexing sequences.

To support this protocol, functions with yield statement are compiled specially as generators. They return a generator object when they are called. The returned object supports the iteration interface with an automatically created __next__() method to resume execution. Generator functions may have a return simply terminates the generation of values by raising a StopIteration exceptions after any normal function exit.

SpiralTrain developer courses

Conveniant Iterator

- **Generator function mainly more convenient to write an iterator :**
  - Don't worry about iterator protocol with `__next__ ()` and `__iter__()`
  - It just works
- **Generators versus iterators :**
  - Generator function is slightly different than an object that supports iteration
  - Generator is a one-time operation :
    - Can iterate over the generated data once
    - If you want to do it again, you have to call the generator function again
  - This is different than a list :
    - You can iterate over al list as many times as you want

The net effect is that generator functions, coded as def statements containing yield statement, are automatically made to support the iteration protocol and thus may be used any iteration context to produce results over time and on demand. Let's look at the interactive example below:

```
>>> def create_counter(n):
            print('create_counter()')
            while True:
                        yield n
                        print('increment n')
                        n += 1
>>> c = create_counter(2)
>>> c
<generator object create_counter at 0x03004B48>
>>> next(c)
create_counter()
2
>>> next(c)
increment n
3
>>> next(c)
increment n
4
>>>
```

SpiralTrain developer courses

Generator Expression

- **Generated version of a list comprehension :**

```
li = [1,2,3,4]
# generator expression, watch the parenthesis
ge = (2*x for x in li)
print (ge)       # <generator object <genexpr> at 0x0085B4E0>
print (next(ge)) # prints 2
for i in ge:      # prints 4, 6 and 8
    print (i)
```

- **Loops over sequence of items and applies operation to each :**
  - However, results are produced one at a time using a generator
- **Important differences from a list comprehension :**
  - Generator Expression does not construct a list
  - Its only useful purpose is iteration and once consumed it can't be reused

```
# list comprehension, watch the block brackets
lc = [2*x for x in li]
print (lc)  # [2, 4, 6, 8]
```



Here are the things happening in the code:

The presence of the yield keyword in create_counter() means that this is not a normal function. It is a special kind of function which generates values one at a time. We can think of it as a resumable function. Calling it will return a generator that can be used to generate successive values of n.

To create an instance of the create_counter() generator, just call it like any other function. Note that this does not actually execute the function code. We can tell this because the first line of the create_counter() function calls print(), but nothing was printed from the line:

>>> c = create_counter(2)

The create_counter() function returns a generator object.

The next() function takes a generator object and returns its next value. The first time we call next() with the counter generator, it executes the code in create_counter() up to the first yield statement, then returns the value that was yielded. In this case, that will be 2, because we originally created the generator by calling create_counter(2).

Repeatedly calling next() with the same generator object resumes exactly where it left off and continues until it hits the next yield statement. All variables, local state, &c. are saved on yield and restored on next(). The next line of code waiting to be executed calls print(), which prints increment n. After that, the statement n += 1. Then it loops through the while loop again, and the first thing it hits is the statement yield n, which saves the state of everything and returns the current value of n (now 3).

The second time we call next(c), we do all the same things again, but this time n is now 4.

Since create_counter() sets up an infinite loop, we could theoretically do this forever, and it would just keep incrementing n and spitting out values.

Generator Expression Syntax

- **General syntax :**

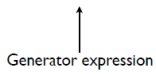
```
(expression for i in s if cond1
            for j in t if cond2
            ...
            if condfinal)
```

- **Resolves to :**

```
for i in s:
    if cond1:
        for j in t:
            if cond2:
                ...
                if condfinal: yield expression
```

- **If used as a single function argument :**
  - Parenthesis on a generator expression can dropped

```
sum(x*x for x in s)
```



The generator function in the following example generated the cubics of numbers over time:

```
>>> def cubic_generator(n):
            for i in range(n):
                        yield i ** 3


>>>
```

The function yields a value and so returns to its caller each time through the loop. When it is resumed, its prior state is restored and control picks up again after the yield statement. When it's used in a for loop, control returns to the function after its yield statement each time through the loop:

```
>>> for i in cubic_generator(5):
            print(i,  end=' : ')  # Python 3.0
        #print i,                 # Python 2.x
0 : 1 : 8 : 27 : 64 :
>>>
```

If we use return instead of yield, the result is:

```
>>> def cubic_generator(n):
            for i in range(n):
                        return i ** 3
>>> for i in cubic_generator(5):
            print(i,  end=' : ')  #Python 3.0
Traceback (most recent call last):
  File "", line 1, in
    for i in cubic_generator(5):
TypeError: 'int' object is not iterable
```

SpiralTrain developer courses

Building Blocks

- **Generator functions :**

```
def countdown(n):
   while n > 0:
      yield n
   n -= 1
```

- **Generator expressions :**

```
squares = (x*x for x in s)
```

- **In both cases :**
  - Get object that generates values which are typically consumed in a for loop

Here is an example of using generator and yield.

```
>>> # Fibonacci version 1
>>> def fibonacci():
            Limit = 10
            count = 0
            a, b = 0, 1
            while True:
                        yield a
                        a, b = b, a+b
                        if (count == Limit):
                                    break
                        count += 1
>>>
>>> for n in fibonacci():
            print(n, end=' ')
0 1 1 2 3 5 8 13 21 34 55
>>>
```

Because generators preserve their local state between invocations, they're particularly well-suited for complicated, stateful iterators, such as **fibonacci numbers**. The generator returning the Fibonacci numbers using Python's yield statement can be seen below.

Here is another version of Fibonacci:

SpiralTrain developer courses

Programming Problem

- **Find out how many bytes of data were transferred :**
  - Sum up last column of data in huge Apache web server log

```
81.107.39.38 - ... "GET /ply/ HTTP/1.1" 200 7587
81.107.39.38 - ... "GET /favicon.ico HTTP/1.1" 404 133
81.107.39.38 - ... "GET /ply/bookplug.gif HTTP/1.1" 200 23903
81.107.39.38 - ... "GET /ply/ply.html HTTP/1.1" 200 97238
81.107.39.38 - ... "GET /ply/example.html HTTP/1.1" 200 2359
66.249.72.134 - ... "GET /index.html HTTP/1.1" 200 4447
```

- **Each line of log looks like this :**

```
81.107.39.38 - ... "GET /ply/ply.html HTTP/1.1" 200 97238
```

- **Number of bytes is the last column :**

```
bytestr = line.rsplit(None,1)[1]
```

- **Is either a number or a missing value (-) :**

```
81.107.39.38 - ... "GET /ply/ HTTP/1.1" 304 -
```



```
>>> # Fibonacci version 2
>>> def fibonacci(max):
    a, b = 0, 1             (1)
    while a < max:
        yield a             (2)
        a, b = b, a + b     (3)
```

Simple summary for this version:

It starts with 0 and 1, goes up slowly at first, then more and more rapidly. To start the sequence, we need two variables: a starts at 0, and b starts at 1.

a is the current number in the sequence, so yield it.

b is the next number in the sequence, so assign that to a, but also calculate the next value a + b and assign that to b for later use. Note that this happens in parallel; if a is 3 and b is 5, then a, b = b, a + b will set a to 5 (the previous value of b) and b to 8 (the sum of the previous values of a and b).

```
>>> for n in fibonacci(500):
            print(n, end=' ')
0 1 1 2 3 5 8 13 21 34 55 89 144 233 377
>>>
>>> list(fibonacci(500))
[0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377]
>>>
```

SpiralTrain developer courses

Problem Solutions

- **Non-Generator Solution :**
  - Do a simple for-loop, read-by-line and update a sum :

```
wwwlog = open("access-log")
total = 0
for line in wwwlog:
    bytestr = line.rsplit(None,1)[1]
    if bytestr != '-':
        total += int(bytestr)
print ("Total", total)
```

- **Generator Solution :**
  - Use some generator expressions

```
wwwlog = open("access-log")
bytecolumn = (line.rsplit(None,1)[1] for line in wwwlog)
bytes = (int(x) for x in bytecolumn if x != '-')
print ("Total", sum(bytes))
```

- **Notice lesser code and a completely different programming style**

As we can see from the output, we can use a generator like fibonacci() in a for loop directly. The for loop will automatically call the next() function to get values from the fibonacci() generator and assign them to the for loop index variable (n). Each time through the for loop, n gets a new value from the yield statement in fibonacci(), and all we have to do is print it out. Once fibonacci() runs out of numbers (a becomes bigger than max, which in this case is 500), then the for loop exits gracefully.

This is a useful idiom: pass a generator to the list() function, and it will iterate through the entire generator (just like the for loop in the previous example) and return a list of all the values.

To end the generation of values, functions use either a return with no value or simply allow control to fall off the end of the function body.

To see what's happening inside the for, we can call the generator function directly:

```
>>> x = cubic_generator(5)
>>> x
<generator object cubic_generator at 0x000000000315F678>
>>>
```

SpiralTrain developer courses

Summary : Generators

- **Python for statement can iterate over many kinds of objects :**
  - Iterate over a collection of items, over dictionary given the keys
- **Many functions can consume an iterable object :**
  - Reductions, constructors etc.
- **Iteration protocol implemented by objects you can iterate over :**
  - Objects supporting `__iter__()` and `__next__()` is "iterable“
- **Generators are functions :**
  - Producing sequence of results instead of single value
- **Generator behavior is quite different from normal function :**
  - Calling generator function creates generator object, does not start running function
- **Generator function mainly more convenient to write an iterator :**
  - Don't worry about iterator protocol with `__next__ ()` and `__iter__()`
- **Generator Expression :**
  - Loops over sequence of items and applies operation to each



© copyright : spiraltrain@gmail.com